HPD Intervals, Sequential Stopping, and Exact Confidence Inversion for a Bernoulli Trial

Nahyun Lee

03/16/2025

Contents

1. Introduction 1
2. Background 1
3. Methodology 2
4. R Implementation 4
5. Results 8
6. Conclusion 16

1. Introduction

This project investigates Bayesian inference for a Bernoulli success probability.

The central object is the unknown success probability `p` in a Bernoulli trial. We begin with a `Beta(1,1)` prior, update it with observed successes and failures, and use the posterior distribution to construct **Highest Posterior Density (HPD)** intervals.

The report develops in three steps:

1. compute an exact 95% HPD interval for a binomial proportion,
2. use the HPD width as a sequential stopping rule,
3. invert the resulting stopping distribution to obtain an exact 95% confidence interval for `p`.

This following sections demonstrate the integration of:

$$\text{posterior inference} \to \text{sequential stopping} \to \text{test inversion}.$$

2. Background

2.1 Beta-Binomial Updating

We use the conjugate Beta-Binomial model:

$$p \sim \text{Beta}(1,1), \qquad X \mid p \sim \text{Binomial}(n,p).$$

If we observe `s` successes and `f` failures, then the posterior is

$$p \mid (s, f) \sim \text{Beta}(1 + s, 1 + f).$$

This posterior distribution is the basis for all interval calculations in the report.

2.2 Confidence-Test Duality

The main conceptual point of the third assignment is that confidence regions and significance tests are dual. If a level-`alpha` test fails to reject a parameter value `p`, then that value belongs to the corresponding `1-alpha` confidence region.

This idea is especially useful here because the stopping rule changes the sampling distribution. Under fixed-sample binomial sampling, one would base inference on

$$S_n \sim \text{Binomial}(n, p),$$

with a deterministic sample size `n`. But in the present problem the sample size is itself random:

$$N = \inf\left\{n : \text{width}(\text{HPD}_n) \leq 0.03\right\},$$

so the observed statistic is not simply `S_n`, but the stopped pair `(S_N, F_N)`. Its distribution depends jointly on `p` and on the stopping rule.

Therefore, a standard fixed-sample interval such as the Wald interval is not aligned with the actual data-generating mechanism. Its nominal coverage is derived under a non-random `n`, whereas here inference must be based on the boundary-hitting probabilities induced by the sequential design. For that reason, instead of plugging the stopped data into a fixed-sample formula, we invert the **sequential stopping distribution** itself.

2.3 Brown, Cai, and DasGupta (2001)

Brown, Cai, and DasGupta show that interval estimation for a binomial proportion is more subtle than standard textbook formulas suggest. Their main lessons relevant to this report are:

- the Wald interval is unreliable,
- better intervals come from principled inversion methods,
- the data-generating mechanism matters,
- sequential designs should be analyzed using the actual stopping rule.

That last point motivates the exact inversion approach used in the final section.

3. Methodology

3.1 Exact HPD Interval for a Beta Posterior

To avoid the instability of simulation-based HPD estimates, we compute the shortest posterior interval directly.

The HPD interval is the set of parameter values whose posterior density exceeds a threshold chosen so that the set contains 95% posterior probability mass. When the posterior is unimodal, this interval can be obtained by optimizing over the left-tail probability.

3.2 Sequential Stopping Rule

We now extend the posterior interval calculation to a sequential Bernoulli experiment:

- after each Bernoulli observation,
- update the posterior `Beta(1+s,1+f)`,
- compute the 95% HPD interval,
- stop when the HPD width is at most `0.03`.

The first state `(s,f)` satisfying this condition lies on the stopping boundary. Because the prior is `Beta(1,1)`, the posterior after `(s,f)` is `Beta(1+s,1+f)`, while after `(f,s)` it is `Beta(1+f,1+s)`. These two posteriors are mirror images of each other around `p = 1/2`, so their HPD widths are identical. Consequently, the stopping boundary is symmetric about the diagonal.

This symmetry is therefore not a generic feature of sequential Bernoulli stopping rules. It is a direct consequence of using a symmetric prior centered at `1/2`. If the prior were asymmetric, such as `Beta(2,1)` or `Beta(1,3)`, the boundary would generally no longer be symmetric.

3.3 Boundary Hitting Probabilities

For any point on the stopping boundary, we compute the probability that a Bernoulli process with success probability `p` hits that point first.

If `(s,f)` is a boundary point, then

$$P_p\{\text{stop at } (s,f)\} = C_{s,f} p^s (1-p)^f,$$

where `C_{s,f}` is the total number of admissible paths that stay inside the continuation region until the final step.

These coefficients differ from the standard binomial coefficients because only paths that remain inside the continuation region until the final step are counted.

These coefficients are computed by dynamic programming starting at `(0,0)` and propagating probability mass outward through the continuation region.

Dynamic programming is used to efficiently propagate probability mass across the lattice while preventing expansion beyond the stopping boundary.

3.4 Exact Confidence Interval by Inversion

Suppose the trial stops at

$$(s_{obs}, f_{obs}) = (1359, 2523).$$

The observed boundary index is

$$\hat{p}_{obs} = \frac{1359}{1359 + 2523}.$$

This index coincides with the maximum likelihood estimator of the Bernoulli success probability.

We construct a 95% confidence interval by inverting the sequential distribution:

- the lower endpoint solves `P_p(Index >= observed index) = 0.025`,
- the upper endpoint solves `P_p(Index <= observed index) = 0.025`.

This produces an exact equal-tailed confidence set with respect to the stopped experiment. In other words, the confidence interval is defined by the acceptance region of the sequential test itself, rather than by a large-sample approximation that ignores stopping.

4. R Implementation

The implementation is divided into three layers:

1. numerical utilities and exact HPD calculation,
2. sequential stopping-boundary construction,
3. boundary-hitting probability evaluation.

The detailed code is retained because these functions are the computational core of the report, but the blocks are separated below so that the overall logic is easier to follow.

All reported numerical values are produced by deterministic computation rather than Monte Carlo simulation. The HPD interval endpoints are obtained by `optimize()`, and the confidence interval endpoints are obtained by `uniroot()` with tolerance `1e-8`. Accordingly, the decimal precision reported in the results reflects numerical optimization accuracy rather than simulation noise.

4.1 Numerical Utilities and HPD Computation

```r
# Add two log-probabilities stably on the log scale.
logspace_add <- function(logx, logy) {
  if (is.infinite(logx) && logx < 0) return(logy)
  if (is.infinite(logy) && logy < 0) return(logx)
  m <- max(logx, logy)
  m + log(exp(logx - m) + exp(logy - m))
}

# Compute the exact shortest credible interval for a Beta posterior.
beta_hpd_interval <- function(a, b, cred_mass = 0.95) {
  tail_mass <- 1 - cred_mass

  if (a <= 1 && b > 1) return(c(0, qbeta(cred_mass, a, b)))
  if (b <= 1 && a > 1) return(c(qbeta(tail_mass, a, b), 1))
  if (a <= 1 && b <= 1) return(c(0, 1))

  objective <- function(left_tail) {
    qbeta(left_tail + cred_mass, a, b) - qbeta(left_tail, a, b)
  }

  opt <- optimize(objective, interval = c(0, tail_mass))

  c(
    qbeta(opt$minimum, a, b),
    qbeta(opt$minimum + cred_mass, a, b)
  )
```

```r
}

# HPD interval width at state (successes, failures).
posterior_width <- function(successes, failures, alpha = 0.05) {
  interval <- beta_hpd_interval(1 + successes, 1 + failures, 1 - alpha)
  interval[2] - interval[1]
}
```

4.2 Sequential Boundary Construction

```r
# For each success count in the lower half of the boundary,
# find the smallest failure count that satisfies the stopping rule.
compute_lower_boundary <- function(target_width = 0.03, alpha = 0.05) {
  thresholds <- integer()
  current_failures <- 0L
  successes <- 0L

  repeat {
    current_failures <- max(0L, current_failures - 25L)
    current_width <- 1

    while (current_width > target_width) {
      current_failures <- current_failures + 1L
      current_width <- posterior_width(successes, current_failures, alpha)
    }

    thresholds[successes + 1L] <- current_failures

    if (current_failures <= successes) break
    successes <- successes + 1L
  }

  thresholds
}

# Propagate path mass from (0,0) through the continuation region.
# Once a transition lands on the stopping boundary, record that first-hit mass.
build_boundary_paths <- function(lower_boundary) {
  max_coord <- max(lower_boundary)
  max_small <- length(lower_boundary) - 1L

  is_stopping <- function(s, f) {
    if (s == 0L && f == 0L) return(FALSE)
    smaller <- min(s, f)
    if (smaller > max_small) return(TRUE)
    if (s <= f) {
      f >= lower_boundary[s + 1L]
    } else {
      s >= lower_boundary[f + 1L]
    }
  }
```

```r
log_paths <- matrix(-Inf, nrow = max_coord + 1L, ncol = max_coord + 1L)
log_paths[1, 1] <- 0

boundary_s <- integer()
boundary_f <- integer()
log_hit_coeff <- numeric()
seen <- new.env(parent = emptyenv())

add_boundary <- function(s, f, logv) {
  key <- paste0(s, ",", f)

  if (exists(key, envir = seen, inherits = FALSE)) {
    idx <- get(key, envir = seen)
    log_hit_coeff[idx] <<- logspace_add(log_hit_coeff[idx], logv)
  } else {
    boundary_s <<- c(boundary_s, s)
    boundary_f <<- c(boundary_f, f)
    log_hit_coeff <<- c(log_hit_coeff, logv)
    assign(key, length(log_hit_coeff), envir = seen)
  }
}

for (n in 0:(2L * max_coord - 1L)) {
  s_min <- max(0L, n - max_coord)
  s_max <- min(n, max_coord)
  if (s_min > s_max) next

  for (s in s_min:s_max) {
    f <- n - s

    if (s == 0L && f == 0L) {
      current_log_paths <- 0
    } else {
      if (is_stopping(s, f)) next
      current_log_paths <- log_paths[s + 1L, f + 1L]
      if (!is.finite(current_log_paths)) next
    }

    next_s <- s + 1L
    next_f <- f
    if (next_s <= max_coord) {
      if (is_stopping(next_s, next_f)) {
        add_boundary(next_s, next_f, current_log_paths)
      } else {
        log_paths[next_s + 1L, next_f + 1L] <- logspace_add(
          log_paths[next_s + 1L, next_f + 1L],
          current_log_paths
        )
      }
    }

    next_s <- s
    next_f <- f + 1L
```

```r
      if (next_f <= max_coord) {
        if (is_stopping(next_s, next_f)) {
          add_boundary(next_s, next_f, current_log_paths)
        } else {
          log_paths[next_s + 1L, next_f + 1L] <- logspace_add(
            log_paths[next_s + 1L, next_f + 1L],
            current_log_paths
          )
        }
      }
    }
  }

  data.frame(
    successes = boundary_s,
    failures = boundary_f,
    index = boundary_s / (boundary_s + boundary_f),
    log_hit_coeff = log_hit_coeff
  )
}
```

4.3 Boundary and Pointwise Stopping Probabilities

```r
# Evaluate the full boundary-hitting distribution at a given success probability p.
boundary_probabilities <- function(boundary_df, p) {
  if (p <= 0 || p >= 1) return(rep(0, nrow(boundary_df)))

  exp(
    boundary_df$log_hit_coeff +
      boundary_df$successes * log(p) +
      boundary_df$failures * log1p(-p)
  )
}

# Evaluate the probability of one specific observed stopping point.
point_probability <- function(boundary_df, successes, failures, p) {
  idx <- which(boundary_df$successes == successes & boundary_df$failures == failures)
  if (length(idx) != 1L) stop("Target point not uniquely identified on the boundary.")
  if (p <= 0 || p >= 1) return(0)

  exp(
    boundary_df$log_hit_coeff[idx] +
      successes * log(p) +
      failures * log1p(-p)
  )
}
```

The confidence interval inversion itself is carried out later in the results section, once the exact stopping boundary and its hitting probabilities have been computed.

5. Results

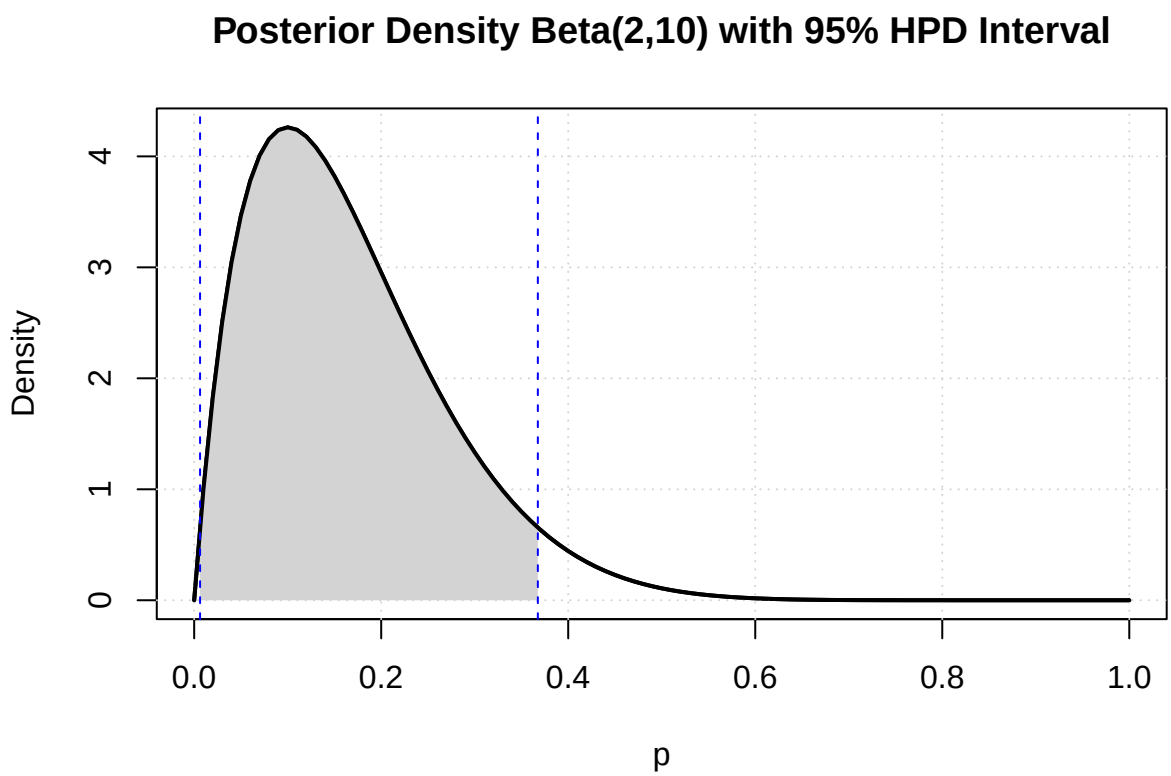
5.1 Exact HPD Interval for a Simple Binomial Example

We first apply the HPD calculation to a simple fixed-sample Bernoulli example, using `1` success and `9` failures.

```r
simple_hpd <- beta_hpd_interval(a = 2, b = 10, cred_mass = 0.95)
cat(sprintf("95%% HPD interval for Beta(2,10): [%.4f, %.4f]\n",
            simple_hpd[1], simple_hpd[2]))
```

```
## 95% HPD interval for Beta(2,10): [0.0063, 0.3675]
```

```r
curve(
  dbeta(x, 2, 10),
  from = 0,
  to = 1,
  lwd = 2,
  xlab = "p",
  ylab = "Density",
  main = "Posterior Density Beta(2,10) with 95% HPD Interval"
)
grid()

x_fill <- seq(simple_hpd[1], simple_hpd[2], length.out = 300)
polygon(
  c(simple_hpd[1], x_fill, simple_hpd[2]),
  c(0, dbeta(x_fill, 2, 10), 0),
  col = "lightgray",
  border = NA
)
curve(dbeta(x, 2, 10), from = 0, to = 1, add = TRUE, lwd = 2)
abline(v = simple_hpd, lty = 2, col = "blue")
```



For this fixed-sample example, the 95% HPD interval is [0.0063, 0.3675]. This provides the basic Bayesian interval machinery used in the rest of the report.

5.2 Sequential Stopping Boundary

```
target_width <- 0.03
alpha <- 0.05

lower_boundary <- compute_lower_boundary(target_width = target_width, alpha = alpha)
boundary_df <- build_boundary_paths(lower_boundary)

target_successes <- 1359L
target_failures <- 2523L
target_index <- target_successes / (target_successes + target_failures)
target_on_boundary <- any(
  boundary_df$successes == target_successes &
    boundary_df$failures == target_failures
)

boundary_plot_x <- c(0:(length(lower_boundary) - 1L), lower_boundary)
boundary_plot_y <- c(lower_boundary, 0:(length(lower_boundary) - 1L))

cat("Number of first-hit boundary points:", nrow(boundary_df), "\n")
```

```
## Number of first-hit boundary points: 4266
```

```r
cat("Largest lower-half boundary failure count:", max(lower_boundary), "\n")
```

```
## Largest lower-half boundary failure count: 2528
```

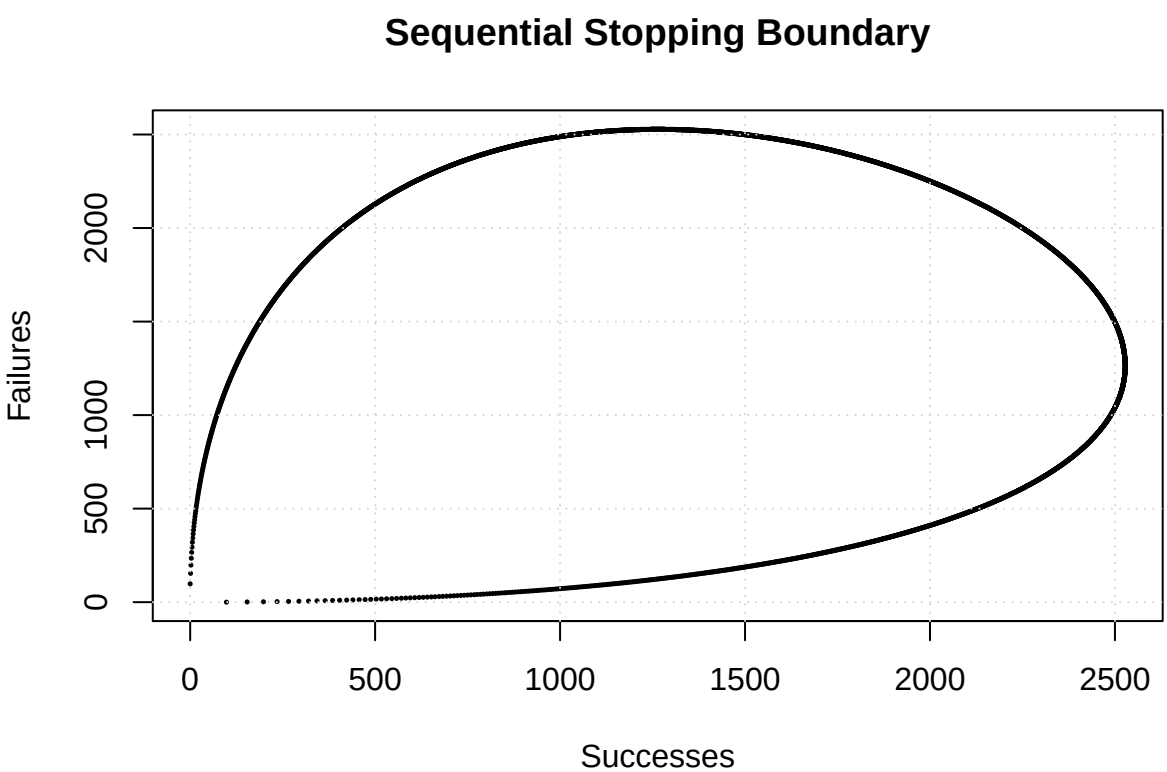
```r
cat("Observed index at (1359, 2523):", round(target_index, 6), "\n")
```

```
## Observed index at (1359, 2523): 0.350077
```

```r
cat("Is (1359, 2523) on the first-hit boundary?", target_on_boundary, "\n")
```

```
## Is (1359, 2523) on the first-hit boundary? TRUE
```

```r
plot(
  boundary_plot_x,
  boundary_plot_y,
  pch = 16,
  cex = 0.35,
  xlab = "Successes",
  ylab = "Failures",
  main = "Sequential Stopping Boundary"
)
grid()
```



The stopping boundary is approximately symmetric around the diagonal and reaches its largest total sample size near `p = 0.5`, where posterior uncertainty is greatest. That symmetry arises from the `Beta(1,1)` prior: exchanging successes and failures leaves the posterior width unchanged, so the stopping rule treats the two sides of the diagonal identically. The large boundary size reflects the irregular geometry of the stopping rule, where many terminal points occur before reaching the maximum trial limit.

5.3 Hitting Probabilities on the Boundary when `p = 1/3`

```r
p0 <- 1 / 3
boundary_df$prob_p_one_third <- boundary_probabilities(boundary_df, p0)
ord <- order(boundary_df$index)

weighted_index_mean <- weighted.mean(boundary_df$index, boundary_df$prob_p_one_third)
weighted_index_sd <- sqrt(
  weighted.mean(
    (boundary_df$index - weighted_index_mean)^2,
    boundary_df$prob_p_one_third
  )
)

zoom_half_width <- max(0.01, 4 * weighted_index_sd)
zoom_limits <- c(
  max(0, weighted_index_mean - zoom_half_width),
  min(1, weighted_index_mean + zoom_half_width)
)

cat("Total boundary probability at p = 1/3:", sum(boundary_df$prob_p_one_third), "\n")
```

```
## Total boundary probability at p = 1/3: 1
```

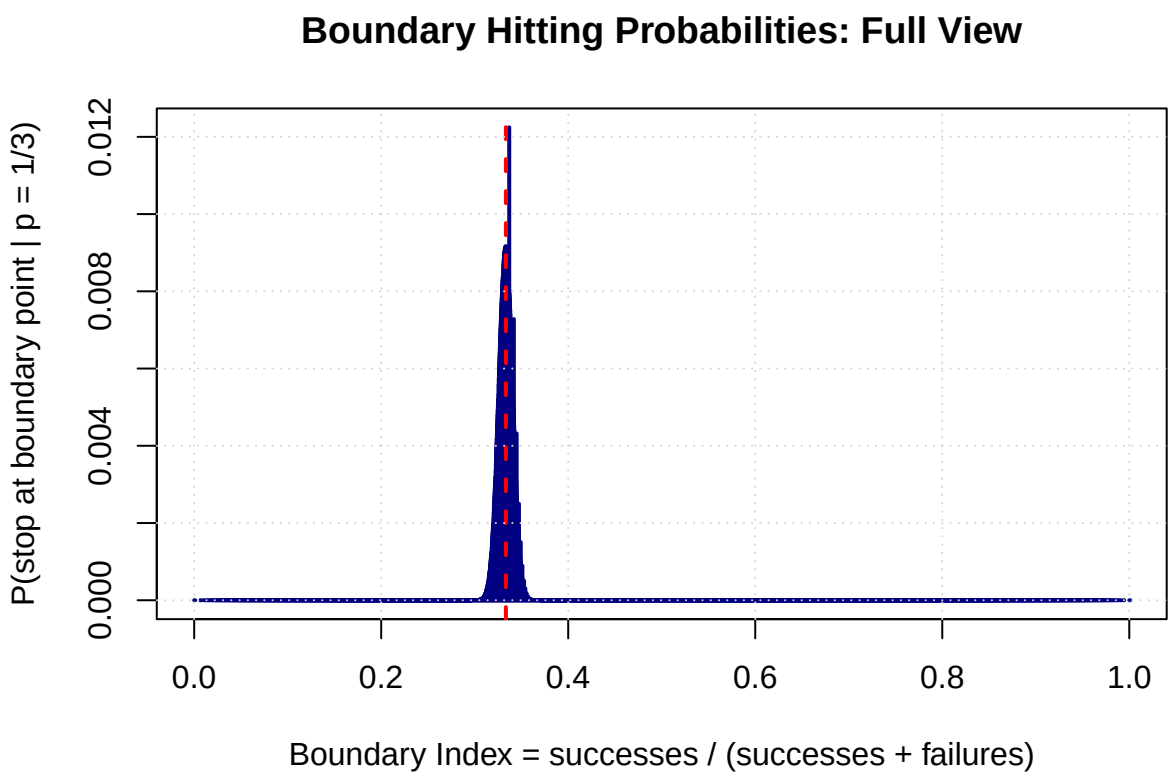
```r
cat("Weighted mean boundary index:", round(weighted_index_mean, 6), "\n")
```

```
## Weighted mean boundary index: 0.333245
```

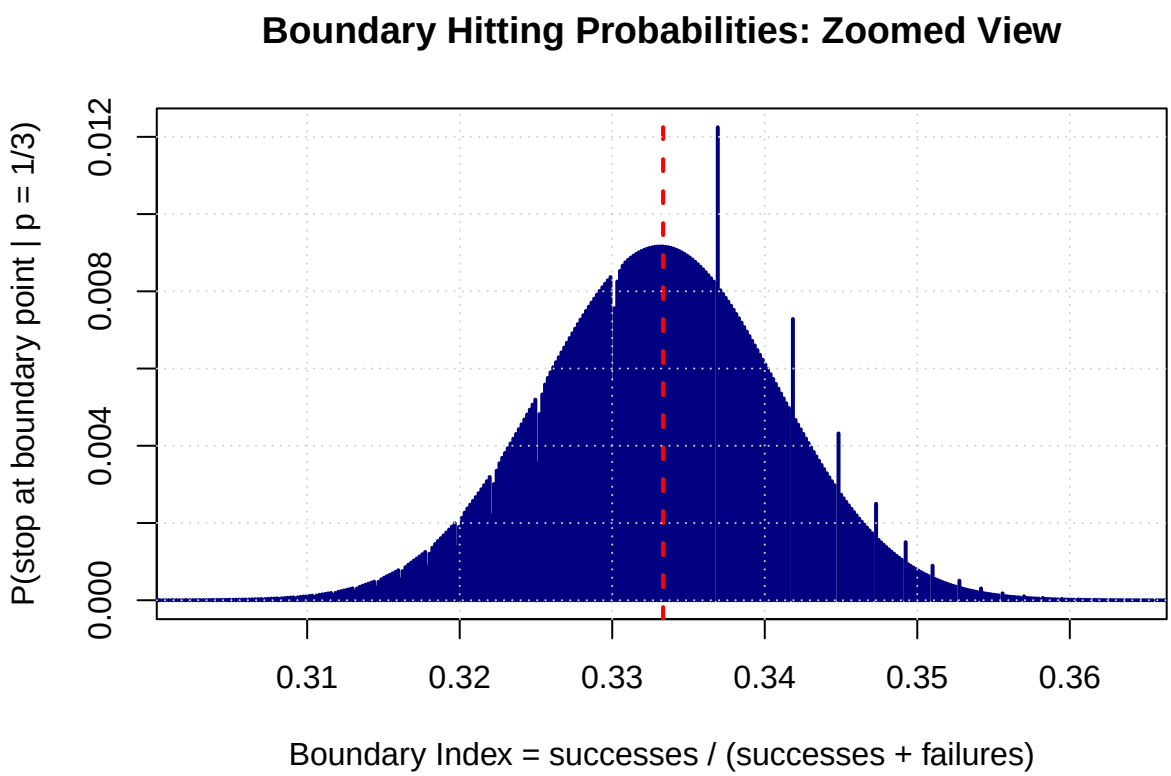
```r
cat("Weighted SD of boundary index:", round(weighted_index_sd, 6), "\n")
```

```
## Weighted SD of boundary index: 0.007663
```

```r
plot(
  boundary_df$index[ord],
  boundary_df$prob_p_one_third[ord],
  type = "h",
  lwd = 2,
  col = "navy",
  xlab = "Boundary Index = successes / (successes + failures)",
  ylab = "P(stop at boundary point | p = 1/3)",
  main = "Boundary Hitting Probabilities: Full View"
)
abline(v = p0, col = "red", lty = 2, lwd = 2)
grid()
```




```r
plot(
  boundary_df$index[ord],
  boundary_df$prob_p_one_third[ord],
  type = "h",
  lwd = 2,
  col = "navy",
  xlim = zoom_limits,
  xlab = "Boundary Index = successes / (successes + failures)",
  ylab = "P(stop at boundary point | p = 1/3)",
  main = "Boundary Hitting Probabilities: Zoomed View"
)
abline(v = p0, col = "red", lty = 2, lwd = 2)
grid()
```



The full view shows that the mass is concentrated very tightly near `1/3`. The zoomed view makes the local structure visible and confirms that the distribution is centered near the true success probability.

5.4 Exact 95% Confidence Interval at the Observed Boundary Point

```
observed_index <- target_index
tail_prob <- alpha / 2

lower_tail_prob <- function(p) {
  probs <- boundary_probabilities(boundary_df, p)
  sum(probs[boundary_df$index <= observed_index + 1e-15])
}

upper_tail_prob <- function(p) {
  probs <- boundary_probabilities(boundary_df, p)
  sum(probs[boundary_df$index >= observed_index - 1e-15])
}

find_root_bracket <- function(values, grid) {
  idx <- which(diff(sign(values)) != 0)[1]
  if (is.na(idx)) stop("Could not find a sign change for the requested root.")
  c(grid[idx], grid[idx + 1L])
}

lower_root_function <- function(p) upper_tail_prob(p) - tail_prob
upper_root_function <- function(p) lower_tail_prob(p) - tail_prob

p_grid <- seq(0.001, 0.999, by = 0.001)
```

```r
lower_bracket <- find_root_bracket(sapply(p_grid, lower_root_function), p_grid)
upper_bracket <- find_root_bracket(sapply(p_grid, upper_root_function), p_grid)

left <- uniroot(lower_root_function, lower_bracket, tol = 1e-8)$root
right <- uniroot(upper_root_function, upper_bracket, tol = 1e-8)$root

cat(sprintf("95%% confidence interval: [%.6f, %.6f]\n", left, right))
```

```
## 95% confidence interval: [0.335148, 0.365319]
```

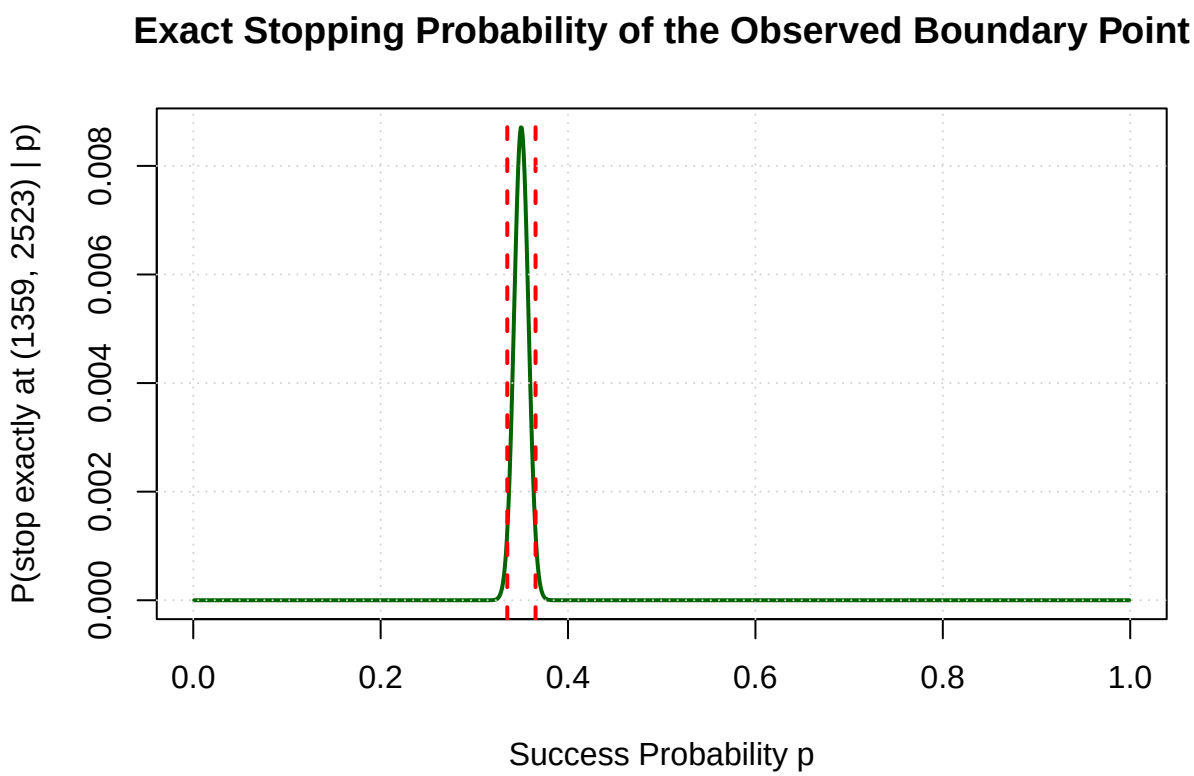
The resulting exact sequential 95% confidence interval is

$$[\,0.335148\,,\,0.365319\,].$$

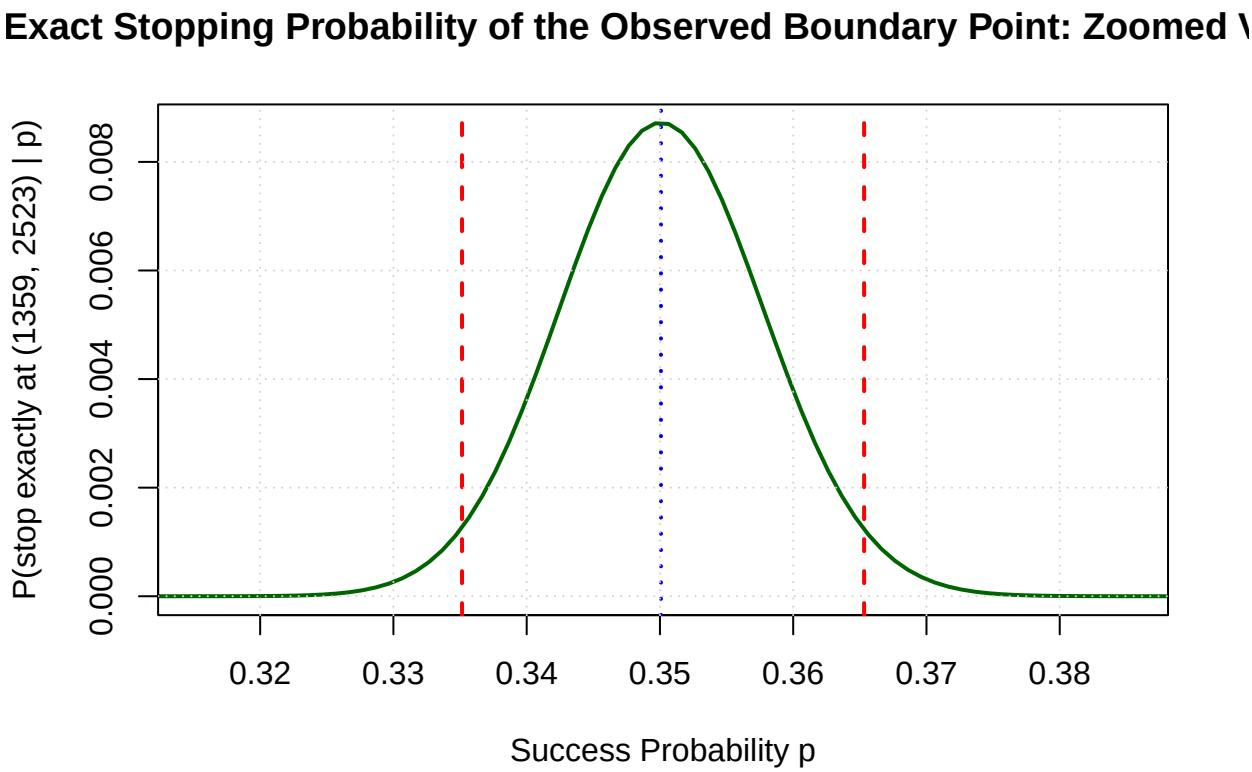
These endpoints are obtained by deterministic root-finding on the exact sequential tail probabilities. In particular, `uniroot()` is applied with tolerance `1e-8`, so the reported six-decimal-place values should be read as numerically stabilized optimization results, not as simulation-based approximations.

5.5 Exact Stopping Probability of the Observed Boundary Point

```r
pp <- seq(0.001, 0.999, length.out = 1000)

point_prob_curve <- sapply(
  pp,
  function(p) point_probability(boundary_df, target_successes, target_failures, p)
)

zoom_curve_limits <- c(
  max(0, left - 0.02),
  min(1, right + 0.02)
)

plot(
  pp,
  point_prob_curve,
  type = "l",
  lwd = 2,
  col = "darkgreen",
  xlab = "Success Probability p",
  ylab = "P(stop exactly at (1359, 2523) | p)",
  main = "Exact Stopping Probability of the Observed Boundary Point"
)
abline(v = c(left, right), lty = 2, lwd = 2, col = "red")
grid()
```




```r
plot(
  pp,
  point_prob_curve,
  type = "l",
  lwd = 2,
  col = "darkgreen",
  xlim = zoom_curve_limits,
  xlab = "Success Probability p",
  ylab = "P(stop exactly at (1359, 2523) | p)",
  main = "Exact Stopping Probability of the Observed Boundary Point: Zoomed View"
)
abline(v = c(left, right), lty = 2, lwd = 2, col = "red")
abline(v = target_index, lty = 3, lwd = 2, col = "blue")
grid()
```



This curve behaves similarly to a likelihood profile because it measures how probable the observed boundary point is as a function of the success probability. It peaks near the observed boundary index and the zoomed view shows how the confidence interval sits around the high-probability region.

6. Conclusion

This report develops one coherent line of analysis for Bernoulli inference. The fixed-sample HPD interval provides the basic Bayesian inference step, the HPD width defines a sequential stopping boundary, and the induced stopping distribution can then be inverted to obtain an exact confidence interval.

The main numerical results are:

- fixed-sample `Beta(2,10)` 95% HPD interval: [0.0063, 0.3675],
- observed boundary point: `(1359, 2523)`,
- observed boundary index: 0.350077,
- exact sequential 95% confidence interval: [0.335148, 0.365319].

Taken together, these results make the confidence-test duality concrete in a sequential Bernoulli setting and show why exact interval inversion is preferable to ad hoc fixed-sample formulas once stopping depends on the data.